

IHK SQL-Master – GA2 Anwendungsentwicklung

فقط برای بخش پایگاه داده / SQL در GA2. قرمز = بسیار پرتکرار در امتحان. سبز = مهم ولی کمی کمتر.

0. Bild insgesamt / نقشه کلی امتحان SQL

:Typische Themen in IHK-GA2 (Anwendungsentwicklung)

- **SELECT mit WHERE und ORDER BY**
- **JOIN über Primär-/Fremdschlüssel (1:n, n:m)**
- **COUNT, SUM, AVG mit GROUP BY**
- **„LEFT JOIN: „ohne Bestellung“, „ohne Teilnahme**
- Begriffe: Primärschlüssel, Fremdschlüssel, 1:1, 1:n, n:m, Zwischentabelle

در GA2 معمولاً از تو می خواهند: - با SELECT چند جدول را بخوانی. - با WHERE فیلتر کنی. - با JOIN رابطه ها را استفاده کنی. - با GROUP BY و COUNT/SUM/AVG آمار بسازی (مثل تعداد Bestellung pro Kunde).

ترفند کلی: هر سوال SQL را به این سؤالات ذهنی بشکن: (۱) از کدام جدولها باید بخوانم؟ (۲) برای وصل کردن جدولها، کلیدهای FK / PK کدامند؟ (۳) دقیقاً چه شرطهای فیلتر باید برآورده شود؟ (۴) آیا یک سطر در خروجی می خواهم یا چند سطر (Aggregat / GROUP BY یا بدون آن)؟

1. Tabellen, Primärschlüssel, Fremdschlüssel

1.1 Primärschlüssel / Fremdschlüssel

Primärschlüssel (PK): eindeutig, nicht NULL, identifiziert einen Datensatz

Fremdschlüssel (FK): verweist auf den Primärschlüssel einer anderen Tabelle

PK = هویت اصلی رکورد در جدول (مثلاً Kundennummer). FK = ستونی در جدول دیگر که به این PK اشاره می‌کند (مثلاً KundenNr در جدول Bestellung). در JOIN تقریباً همیشه از رابطه $FK = PK$ استفاده می‌کنی.

1.2 Kardinalität

1:1 selten, z. B. Person – Personalausweis

1:n sehr häufig, z. B. Kunde – Bestellung. Ein Kunde, viele Bestellungen

n:m via Zwischentabelle, z. B. Schüler – Kurs

۱:ن = یک رکورد در جدول A می‌تواند چند رکورد در جدول B داشته باشد، ولی برعکس نه. $n:m$ = دو طرف می‌توانند چندتایی باشند، که در عمل با یک جدول وسط حل می‌شود.

ترفند تشخیص از روی ER-Diagram: علامت Krähenfuß (چنگال) یعنی «چند». - جدول وسط در $n:m$ معمولاً فقط دو FK دارد (به دو جدول اصلی) و خودش PK ترکیبی از این دو است.

2. SELECT – انتخاب ستون‌ها و سطرها

2.1 Grundform

```
SELECT Spalten  
FROM   Tabelle  
WHERE  Bedingung  
ORDER BY Spalte [ASC|DESC];
```

چهار جزء اصلی: - **SELECT**: کدام ستون‌ها را می‌خواهی؟ - **FROM**: از کدام جدول‌ها؟ - **WHERE**: شرط روی سطرها. - **ORDER BY**: مرتب‌سازی خروجی.

اگر در سوال نوشت: «geben Sie alle Kunden mit Name und Ort aus, sortiert nach Name» مستقیم تبدیل می‌شود به:

```
SELECT name, ort  
FROM   kunde  
ORDER BY name ASC;
```

تर्फند: - اگر «alle Spalten» لازم نیست، همیشه فقط ستون‌های لازم را بنویس (نه SELECT *). - اگر در امتحان محدودیت زمانی داری، اول اسکلت SELECT-FROM-WHERE-ORDER BY را بنویس، بعد جزئیات را پر کن، تا حداقل ساختار را داشته باشی.

3. WHERE – فیلتر کردن سطرها

3.1 Operatoren

```
= < > <= >= <> (ungleich)
AND OR NOT
BETWEEN a AND b
IN (Liste)
LIKE 'M%'      -- beginnt mit M
LIKE '%M%'     -- enthält M
IS NULL / IS NOT NULL
```

- **AND** = همه شرطها باید برقرار باشند. - **OR** = کافی است یکی برقرار باشد. - **BETWEEN** شامل هر دو سر است (a و b). - **LIKE** با % برای جستجوی جزئی (مثلاً شهرهایی که با „Ber“ شروع می‌شوند).

در سوالهای IHK خیلی رایج است: - «Name beginnt mit 'A'» → **LIKE** - «Preis zwischen 10 und 50 Euro» → **BETWEEN 10 AND 50** - «Kunden ohne Telefonnummer» → **telefon IS NULL** - «'A%'».

تर्फند چهارگزینه‌ای: - اگر در صورت سؤال «zwischen X und Y (inklusive)» نوشته، ولی در گزینه‌ها از < و > خالی استفاده شده، مشکوک باش؛ معمولاً جواب صحیح همان گزینه‌ای است که **BETWEEN** یا <= / >= دارد.

4. JOIN – ترکیب جدول‌ها (۱:ن)

4.1. Inner Join (klassisch)

```
SELECT k.kunden_nr, k.name, b.bestell_nr, b.datum
FROM   kunde k
JOIN   bestellung b
      ON k.kunden_nr = b.kunden_nr;
```

این الگو برای همهٔ ۱:نهایست: مشتری-سفارش، Firma-Mitarbeiter، Lehrer-Kurs و ... یک طرف PK دارد، طرف دیگر FK. شرط JOIN همان PK = FK است.

گاهی در امتحان از سینتکس قدیمی استفاده می‌کنند:

```
SELECT k.kunden_nr, k.name, b.bestell_nr
FROM   kunde k, bestellung b
WHERE  k.kunden_nr = b.kunden_nr;
```

از نظر منطقی همان است، فقط به جای JOIN در FROM، رابطه در WHERE آمده است.

ترفند: - اگر در صورت سؤال عبارات «alle Bestellungen samt Artikelbezeichnung»، «mit ihren Kunden»، «zugehörige Bestellungen» دیدی، این یعنی از تو JOIN می‌خواهند. - همیشه نام جدول را با Alias کوتاه کن (k, b, a) تا نوشتن شرط‌ها ساده‌تر شود.

n:m mit Zwischentabelle .5

Muster .5.1

.Beispiel: Schüler – Kurs – SchuelerKurs

```
SELECT  s.name, k.titel
FROM    schueler s
JOIN    schueler_kurs sk
      ON s.id = sk.schueler_id
JOIN    kurs k
      ON k.id = sk.kurs_id;
```

برای n:m همیشه سه جدول لازم داریم: - دو جدول اصلی (schueler, kurs) - یک جدول وسط با دو JOIN (schueler_kurs). FK همیشه از اصلی به وسط، و از وسط به اصلی دیگر است.

ترفند: - اگر در ER-Diagram دو جدول را می‌بینی که با Krähenfuß به دو طرف وصل‌اند، و یک جدول وسط که فقط دو FK دارد $n:m \rightarrow$ - در SQL باید هر سه جدول را در FROM / JOIN بیاوری؛ اگر فقط دو تا را بیاوری، بخشی از رابطه را از دست می‌دهی.

GROUP BY و Aggregatfunktionen .6

Funktionen .6.1

```
COUNT(*)      -- Anzahl Zeilen
COUNT(spalte) -- nur nicht-NULL
SUM(spalte)   -- Summe
AVG(spalte)   -- Durchschnitt
MIN(spalte)   -- Minimum
MAX(spalte)   -- Maximum
```

GROUP BY .6.2

```
SELECT k.kunden_nr,
       k.name,
       SUM(b.betrag) AS gesamt_umsatz
FROM   kunde k
JOIN   bestellung b
       ON k.kunden_nr = b.kunden_nr
GROUP BY k.kunden_nr, k.name
ORDER BY gesamt_umsatz DESC;
```

هر جا در صورت سؤال بخواهد «درجه گروهی» مثل «Anzahl Bestellungen je Kunde», «Umsatz pro Kunde» این یعنی باید GROUP BY استفاده کنی، قانون مهم: - هر ستونی که در SELECT است و Aggregat نیست، باید در GROUP BY هم باشد.

ترفند سؤال چهارگزینه‌ای: - گزینه‌ای که ستون غیر-aggregat دارد ولی آن ستون در GROUP BY نیامده، غلط است. - گزینه‌ای که COUNT(*) بدون GROUP BY دارد ولی صورت سؤال «je Kunde» می‌گوید، مشکوک است.

7. الگوهای پرتکرار سؤال SQL

7.1. Liste aller ... mit Bedingung

```
SELECT k.kunden_nr, k.name, k.ort
FROM   kunde k
WHERE  k.plz BETWEEN 10000 AND 19999
ORDER BY k.name;
```

7.2. Anzahl der Bestellungen pro Kunde (mit Mindestumsatz)

```
SELECT k.kunden_nr,
       k.name,
       COUNT(b.bestell_nr) AS anzahl_bestellungen
FROM   kunde k
JOIN   bestellung b
      ON k.kunden_nr = b.kunden_nr
GROUP BY k.kunden_nr, k.name
HAVING COUNT(b.bestell_nr) >= 3;
```

HAVING برای فیلتر کردن گروه‌ها استفاده می‌شود (بعد از GROUP BY)، در حالی که WHERE برای فیلتر کردن سطرهاى منفرد قبل از Group است.

7.3. Kunden ohne Bestellung (LEFT JOIN)

```
SELECT k.kunden_nr, k.name
FROM   kunde k
LEFT JOIN bestellung b
      ON k.kunden_nr = b.kunden_nr
WHERE  b.bestell_nr IS NULL;
```

الگوی کلاسیک «بدون ...»: - LEFT JOIN از سمت اصلی (kunde) به سمت فرعی (bestellung). - در WHERE چک می‌کنی که ستون طرف فرعی NULL باشد → یعنی هیچ رکوردی متناظر وجود ندارد.

8. ترفندهای عمومی برای سوال‌های چهارگزینه‌ای SQL

- اگر سؤال درباره «je Kunde / je Artikel» است ولی گزینه‌ای GROUP BY ندارد، آن گزینه تقریباً همیشه غلط است.
 - اگر صورت سؤال می‌گوید «zwischen X und Y (inklusive)» و در گزینه‌ها از $X \text{ AND } < Y <$ استفاده شده، معمولاً گزینه‌ای که $BETWEEN \ X \ \text{AND} \ Y$ یا $=>/=<$ دارد درست است.
 - در JOIN، اگر جدول وسط (n:m) در FROM / JOIN نیامده باشد، رابطه کامل نیست و پاسخ غلط است.
 - اگر در SELECT ستونی آمده که نه Aggregat است و نه در GROUP BY ذکر شده، جواب از نظر SQL اشتباه است (غالباً پاسخ خطاست).
 - برای «... ohne» معمولاً از `LEFT JOIN ... WHERE ... IS NULL` استفاده می‌شود، نه از INNER JOIN.
 - در WHERE از `NULL =` استفاده کردن غلط است؛ باید `IS NULL` باشد. گزینه‌ای که `NULL =` دارد، تقریباً حتماً غلط است.
- پیشنهاد تمرین: چند سوال از امتحان‌های قدیمی را بردار، گزینه‌ها را بدون نگاه به جواب تحلیل کن و فقط با این ترفندها (GROUP BY، NULL، JOIN، BETWEEN) گزینه‌ها را حذف کن.

IHK SQL-Master – GA2 Anwendungsentwicklung

فقط برای بخش پایگاه داده / SQL در GA2. قرمز = بسیار پرتکرار در امتحان. سبز = مهم ولی کمی کمتر.

0. Bild insgesamt / نقشه کلی امتحان SQL

:Typische Themen in IHK-GA2 (Anwendungsentwicklung)

SELECT mit WHERE und ORDER BY

JOIN über Primär-/Fremdschlüssel (1:n, n:m)

COUNT, SUM, AVG mit GROUP BY

“LEFT JOIN: „ohne Bestellung“, „ohne Teilnahme

Begriffe: Primärschlüssel, Fremdschlüssel, 1:1, 1:n, n:m, Zwischentabelle

در GA2 معمولاً از تو می خواهند: - با SELECT چند جدول را بخوانی. - با WHERE فیلتر کنی. - با JOIN رابطه ها را استفاده کنی. - با GROUP BY و COUNT/SUM/AVG آمار بسازی (مثل تعداد Bestellung pro Kunde).

ترفند کلی: هر سوال SQL را به این سؤالات ذهنی بشکن: (۱) از کدام جدول ها باید بخوانم؟ (۲) برای وصل کردن جدول ها، کلیدهای PK / FK کدام اند؟ (۳) دقیقاً چه شرط های فیلتر باید برآورده شود؟ (۴) آیا یک سطر در خروجی می خواهم یا چند سطر (Aggregat / GROUP BY یا بدون آن)؟

1. Tabellen, Primärschlüssel, Fremdschlüssel

1.1 Primärschlüssel / Fremdschlüssel

Primärschlüssel (PK): eindeutig, nicht NULL, identifiziert einen Datensatz

Fremdschlüssel (FK): verweist auf den Primärschlüssel einer anderen Tabelle

PK = هویت اصلی رکورد در جدول (مثلاً Kundennummer). FK = ستونی در جدول دیگر که به این PK اشاره می‌کند (مثلاً KundenNr در جدول Bestellung). در JOIN تقریباً همیشه از رابطه $FK = PK$ استفاده می‌کنی.

1.2 Kardinalität

1:1 selten, z. B. Person – Personalausweis

1:n sehr häufig, z. B. Kunde – Bestellung. Ein Kunde, viele Bestellungen

n:m via Zwischentabelle, z. B. Schüler – Kurs

۱:ن = یک رکورد در جدول A می‌تواند چند رکورد در جدول B داشته باشد، ولی برعکس نه. $n:m$ = دو طرف می‌توانند چندتایی باشند، که در عمل با یک جدول وسط حل می‌شود.

ترفند تشخیص از روی ER-Diagram: علامت Krähenfuß (چنگال) یعنی «چند». - جدول وسط در $n:m$ معمولاً فقط دو FK دارد (به دو جدول اصلی) و خودش PK ترکیبی از این دو است.

2. SELECT – انتخاب ستون‌ها و سطرها

2.1 Grundform

```
SELECT Spalten
FROM   Tabelle
WHERE  Bedingung
ORDER BY Spalte [ASC|DESC];
```

چهار جزء اصلی: - **SELECT**: کدام ستون‌ها را می‌خواهی؟ - **FROM**: از کدام جدول‌ها؟ - **WHERE**: شرط روی سطرها. - **ORDER BY**: مرتب‌سازی خروجی.

اگر در سوال نوشت: «geben Sie alle Kunden mit Name und Ort aus, sortiert nach Name» مستقیم تبدیل می‌شود به:

```
SELECT name, ort
FROM   kunde
ORDER BY name ASC;
```

ترفند: - اگر «alle Spalten» لازم نیست، همیشه فقط ستون‌های لازم را بنویس (نه SELECT *). - اگر در امتحان محدودیت زمانی داری، اول اسکلت SELECT-FROM-WHERE-ORDER BY را بنویس، بعد جزئیات را پر کن، تا حداقل ساختار را داشته باشی.

3. WHERE – فیلتر کردن سطرها

3.1 Operatoren

```
= < > <= >= <> (ungleich)
AND OR NOT
BETWEEN a AND b
IN (Liste)
LIKE 'M%' -- beginnt mit M
LIKE '%M%' -- enthält M
IS NULL / IS NOT NULL
```

- **AND** = همه شرطها باید برقرار باشند. - **OR** = کافی است یکی برقرار باشد. - **BETWEEN** شامل هر دو سر است (a و b). - **LIKE** با % برای جستجوی جزئی (مثلاً شهرهایی که با „Ber“ شروع می‌شوند).

در سوالهای IHK خیلی رایج است: - «Name beginnt mit 'A'» → **LIKE** - «Preis zwischen 10 und 50 Euro» → **BETWEEN 10 AND 50** - «Kunden ohne Telefonnummer» → **telefon IS NULL** - «'A%'».

ترفند چهارگزینه‌ای: - اگر در صورت سؤال «zwischen X und Y (inklusive)» نوشته، ولی در گزینه‌ها از < و > خالی استفاده شده، مشکوک باش؛ معمولاً جواب صحیح همان گزینه‌ای است که **BETWEEN** یا <= / >= دارد.

4. JOIN – ترکیب جدول‌ها (۱:ن)

4.1. Inner Join (klassisch)

```
SELECT k.kunden_nr, k.name, b.bestell_nr, b.datum
FROM   kunde k
JOIN   bestellung b
      ON k.kunden_nr = b.kunden_nr;
```

این الگو برای همهٔ ۱:نهایست: مشتری-سفارش، Firma-Mitarbeiter، Lehrer-Kurs و ... یک طرف PK دارد، طرف دیگر FK. شرط JOIN همان PK = FK است.

گاهی در امتحان از سینتکس قدیمی استفاده می‌کنند:

```
SELECT k.kunden_nr, k.name, b.bestell_nr
FROM   kunde k, bestellung b
WHERE  k.kunden_nr = b.kunden_nr;
```

از نظر منطقی همان است، فقط به جای JOIN در FROM، رابطه در WHERE آمده است.

ترفند: - اگر در صورت سؤال عبارات «alle Bestellungen samt Artikelbezeichnung»، «mit ihren Kunden»، «zugehörige Bestellungen» دیدی، این یعنی از تو JOIN می‌خواهند. - همیشه نام جدول را با Alias کوتاه کن (k, b, a) تا نوشتن شرط‌ها ساده‌تر شود.

n:m mit Zwischentabelle .5**Muster .5.1**

.Beispiel: Schüler – Kurs – SchuelerKurs

```
SELECT  s.name, k.titel
FROM    schueler s
JOIN    schueler_kurs sk
      ON s.id = sk.schueler_id
JOIN    kurs k
      ON k.id = sk.kurs_id;
```

برای n:m همیشه سه جدول لازم داریم: - دو جدول اصلی (schueler, kurs) - یک جدول وسط با دو JOIN (schueler_kurs). FK همیشه از اصلی به وسط، و از وسط به اصلی دیگر است.

ترفند: - اگر در ER-Diagram دو جدول را می‌بینی که با Krähenfuß به دو طرف وصل‌اند، و یک جدول وسط که فقط دو FK دارد $n:m \rightarrow$ - در SQL باید هر سه جدول را در FROM / JOIN بیاوری؛ اگر فقط دو تا را بیاوری، بخشی از رابطه را از دست می‌دهی.

GROUP BY و Aggregatfunktionen .6

Funktionen .6.1

```
COUNT(*)      -- Anzahl Zeilen
COUNT(spalte) -- nur nicht-NULL
SUM(spalte)   -- Summe
AVG(spalte)   -- Durchschnitt
MIN(spalte)   -- Minimum
MAX(spalte)   -- Maximum
```

GROUP BY .6.2

```
SELECT k.kunden_nr,
       k.name,
       SUM(b.betrag) AS gesamt_umsatz
FROM   kunde k
JOIN   bestellung b
       ON k.kunden_nr = b.kunden_nr
GROUP BY k.kunden_nr, k.name
ORDER BY gesamt_umsatz DESC;
```

هر جا در صورت سؤال بخواهد «درجه گروهی» مثل «Anzahl Bestellungen je Kunde», «Umsatz pro Kunde» این یعنی باید GROUP BY استفاده کنی، قانون مهم: - هر ستونی که در SELECT است و Aggregat نیست، باید در GROUP BY هم باشد.

ترفند سؤال چهارگزینه‌ای: - گزینه‌ای که ستون غیر-aggregat دارد ولی آن ستون در GROUP BY نیامده، غلط است. - گزینه‌ای که COUNT(*) بدون GROUP BY دارد ولی صورت سؤال «je Kunde» می‌گوید، مشکوک است.

7. الگوهای پرتکرار سؤال SQL

7.1. Liste aller ... mit Bedingung

```
SELECT k.kunden_nr, k.name, k.ort
FROM kunde k
WHERE k.plz BETWEEN 10000 AND 19999
ORDER BY k.name;
```

7.2. Anzahl der Bestellungen pro Kunde (mit Mindestumsatz)

```
SELECT k.kunden_nr,
       k.name,
       COUNT(b.bestell_nr) AS anzahl_bestellungen
FROM kunde k
JOIN bestellung b
  ON k.kunden_nr = b.kunden_nr
GROUP BY k.kunden_nr, k.name
HAVING COUNT(b.bestell_nr) >= 3;
```

HAVING برای فیلتر کردن گروه‌ها استفاده می‌شود (بعد از GROUP BY)، در حالی که WHERE برای فیلتر کردن سطرهاى منفرد قبل از Group است.

7.3. Kunden ohne Bestellung (LEFT JOIN)

```
SELECT k.kunden_nr, k.name
FROM kunde k
LEFT JOIN bestellung b
  ON k.kunden_nr = b.kunden_nr
WHERE b.bestell_nr IS NULL;
```

الگوی کلاسیک «بدون ...»: - LEFT JOIN از سمت اصلی (kunde) به سمت فرعی (bestellung). - در WHERE چک می‌کنی که ستون طرف فرعی NULL باشد → یعنی هیچ رکوردی متناظر وجود ندارد.

8. ترفندهای عمومی برای سوال‌های چهارگزینه‌ای SQL

- اگر سؤال دربارهٔ «je Kunde / je Artikel» است ولی گزینه‌ای GROUP BY ندارد، آن گزینه تقریباً همیشه غلط است.
 - اگر صورت سؤال می‌گوید «zwischen X und Y (inklusive)» و در گزینه‌ها از $X \text{ AND } < Y <$ استفاده شده، معمولاً گزینه‌ای که $\text{BETWEEN } X \text{ AND } Y$ یا $=>/=<$ دارد درست است.
 - در JOIN، اگر جدول وسط (n:m) در FROM / JOIN نیامده باشد، رابطه کامل نیست و پاسخ غلط است.
 - اگر در SELECT ستونی آمده که نه Aggregat است و نه در GROUP BY ذکر شده، جواب از نظر SQL اشتباه است (غالباً پاسخ خطاست).
 - برای «... ohne» معمولاً از `LEFT JOIN ... WHERE ... IS NULL` استفاده می‌شود، نه از INNER JOIN.
 - در WHERE از `NULL =` استفاده کردن غلط است؛ باید `IS NULL` باشد. گزینه‌ای که `NULL =` دارد، تقریباً حتماً غلط است.
- پیشنهاد تمرین: چند سوال از امتحان‌های قدیمی را بردار، گزینه‌ها را بدون نگاه به جواب تحلیل کن و فقط با این ترفندها (GROUP BY، NULL، JOIN، BETWEEN) گزینه‌ها را حذف کن.